

**Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Ciudad Juárez**



**Tecnológico
de Monterrey**

Introducción a la cadena de valor de los semiconductores

Implementación del Protocolo UART en Verilog y Síntesis con TinyTapeout

Victoria Fierro Aveytia - A01247288

Blanca Abigail Méndez Delgado - A01247329

Daniel Octavio Marín Hernández - A01247291

Grupo 301

14 de junio 2026

Índice

I. Introducción.....	2
II. Análisis y descripción del problema.....	4
III. Desarrollo de la propuesta de solución.....	5
A. UART TX.....	5
B. UART RX.....	7
IV. Simulaciones.....	8
V. Funcionamiento de los módulos.....	8
VI. Conclusiones.....	9
VII. Referencias.....	10
VIII. Apéndice A.....	11
IX. Apéndice B.....	11
X. Apéndice C.....	12
XI. Apéndice D.....	14
XII. Apéndice E.....	18

I. Introducción

El dispositivo conocido como Universal Asynchronous Receiver Transmitter (UART), aplica el protocolo de comunicación serial RS-232 de manera práctica (Sunwoldb, 2024). Este estándar es el que define qué niveles de voltajes especifican una lógica de 0 o 1 en la comunicación del circuito UART (Sunwoldb, 2024).

El desarrollo del Recommended Standard 232 o estándar RS-232, se debe a la necesidad de transmisión de datos serial, y ha sido el protocolo para comunicación serial por excelencia desde su creación (Liang, 2024). Aunque se han desarrollado tecnologías más avanzadas para la transmisión y recepción de datos, el RS-232 sigue siendo relevante gracias a su simplicidad (Liang, 2024).

Este protocolo establece conexiones punto a punto, es decir un transmisor se conecta a un receptor, con una velocidad de transmisión (baud rate) y longitud en bits de datos iguales (Liang, 2024). En la figura 1, se puede observar la configuración de conexiones RS-232 más común, tiene 9 pines, y normalmente se utilizan para computadoras (Weis, 2026). En la imagen, podemos ver un cable transmisor (male/macho), y un receptor (female/hembra).

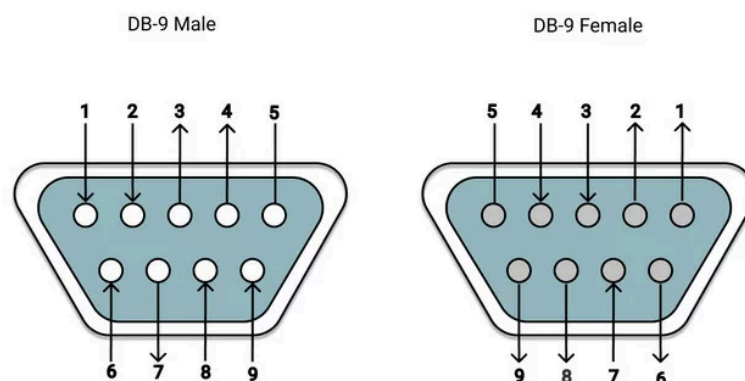


Figura 2. Pines estándar RS-232 (Weis, 2026).

A continuación se desglosan los pines:

- Pin 1 - DCD ← Detección de portadora de datos (Weis, 2026).
- Pin 2 - $R_x D$ ← Recepción de datos (Weis, 2026).
- Pin 3 - $T_x D$ → Transmisión de datos (Weis, 2026).
- Pin 4 - DTR → Terminal de datos listo (Weis, 2026).
- Pin 5 - $0V/COM$ – $0V$ o masa del sistema (Weis, 2026).
- Pin 6 - DSR ← Equipo de datos listo (Weis, 2026).
- Pin 7 - RTS → Solicitud de envío (Weis, 2026).
- Pin 8 - CTS ← Listo para enviar (Weis, 2026).
- Pin 9 - RI ← Indicador de llamada (Weis, 2026).

La comunicación establecida es asincrónica, lo que significa que la transmisión y recepción de datos no es guiada por un reloj, sino que dependen del baud rate en común, el cual también determina la cantidad de bits transmitidos por segundo (Liang, 2024). La transmisión ocurre secuencialmente, desde el bit menos significativo, y termina con bits de stop (figura 2) (Liang, 2024).

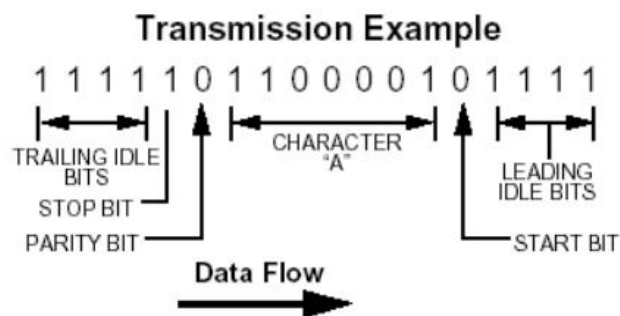


Figura 2. Ejemplo de transmisión de datos con el estándar RS-232 (Liang, 2024).

El estándar RS-232, define a todo dato como bipolar, típicamente, indica una condición de encendido (estado 0) cuando se alcanzan voltajes de entre 3 V y 12 V,

mientras que, sus contrapartes negativas -3 V y -12 V indican una condición de apagado (estado 1) (Weis, 2026). El RS-232 acepta un rango de voltaje de -25 V a 25 V, cuando se reciben voltajes más pequeños, se reduce el rango general de transmisión y recepción (Weis, 2026).

Este tipo de comunicación, puede ser modelado por una máquina de estados finitos (FSM por sus siglas en inglés), un modelo matemático computacional, que permite cambiar entre diferentes estados dadas ciertas condiciones, también conocidas como transiciones (Academia Lab, 2026).

Una FSM contiene estados, que se definen como las posiciones por las que pasa la señal estudiada (Fisicotrónica, 2024). Además, pueden recibir entradas y eventos, los cuales interactúan con el objeto estudiado, pueden alterarlo, e incluso hacerlo cambiar de estado, la diferencia principal entre estos, es que una entrada es externa, mientras que un evento puede ser interno o externo a la FSM (Fisicotrónica, 2024). Finalmente, estas máquinas típicamente tienen señales de salida, determinadas por la funcionalidad de estas (Fisicotrónica, 2024). En este proyecto se realizan 2 máquinas de estado, UART T_x y UART R_x .

II. Análisis y descripción del problema

Como anteriormente mencionado, el protocolo UART facilita la transmisión de datos de manera asíncrona. Esta actividad, requirió el diseño de una máquina de estados que permita este tipo de comunicación, en el lenguaje de programación Verilog. A la hora de realizar el código, se pidió originalidad, y comentarios para facilitar la legibilidad de este.

Para desarrollar la máquina de estados, se crearon 2 módulos, uno para la transmisión de datos, y otro para recibirlos, que luego fueron combinados. La integración con LibreOffice, implicó el ajuste de detalles en el diseño, como la operación del reloj base y los puertos. Finalmente, se sintetizó el diseño utilizando herramientas de GoogleColab, se generaron archivos de prueba y se verificó la viabilidad de la fabricación del diseño.

III. Desarrollo de la propuesta de solución

En las figuras 3 y 4, se muestran los diagramas de estados que se utilizaron para guiar la construcción de las máquinas de estado requeridas. Además, se adjuntan las tablas 1 y 2, las cuales desglosan y explican las señales de entrada, salida y estados de cada máquina.

A. UART TX

- Diagrama de flujo

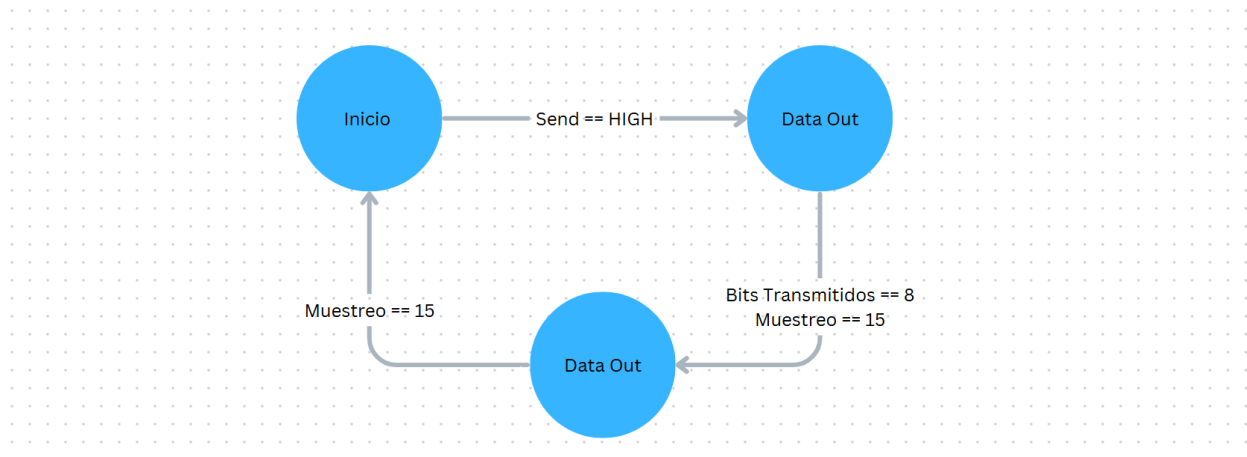


Figura 3. Diagrama de estados $UART T_X$.

- **Tabla de entradas y salidas.**

Tabla 1. Tabla de Señales para $UART T_X$.

	Señal	Descripción
Entradas	clk, rst	Reloj y reset del sistema
	[7:0] dataIn	Dato de 8 bits a transmitir
	send	Señal para iniciar la transmisión
	clkEn	Activa el clk (baud rate x16)
Salidas	tx	Salida a RX
	busy	Indica que la transmisión está en curso
Estados	estadoInicio	Espera la señal de envío
	estadoDataIn	Transmite los 8 bits de datos
	estadoStop	Envía el stop bit

- **Protocolos de cambio de estado**

En el módulo $UART T_X$ existen distintos protocolos para cambiar de estado. A continuación se presenta una lista de estos:

- Ocurre un cambio de estado de estadoInicio a estadoDataIn cuando la señal de send se activa. Esto indica que hay un dato listo para transmitir.
- De estadoDataIn a estadoStop cuando se han transmitido los 8 bits completos.
- De estadoStop a estadoInicio cuando se envía el bit de parada, lo que finaliza la transmisión.

B. UART RX

- Diagrama de flujo

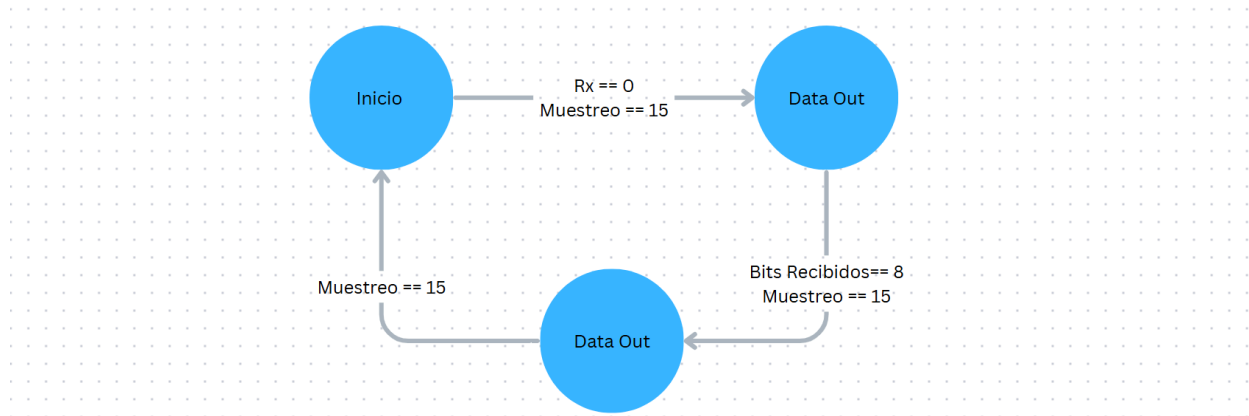


Figura 4. Diagrama de estados $UART R_X$.

- Tabla de entradas y salidas.

Tabla 2. Tabla de Señales para $UART T_X$.

	Señal	Descripción
Entradas	clk, rst	Reloj y reset del sistema
	rx	Serial input
	rdyClr	Reinicia la salida rdy
	clkEn	Activa el clk (baud rate x16)
Salidas	rdy	Salida a TX
	[7:0] dataOut	Outputs recibidos acomodados de manera paralela.
Estados	estadoInicio	Espera el bit de inicio
	estadoDataOut	Captura los 8 bits de datos
	estadoStop	Verifica el stop bit

- **Protocolo de cambio estados**

Para el cambio de estados en el módulo de UART R_x existen 3 protocolos principales, correspondientes a cada estado. A continuación, se listan dichos protocolos, en el orden esperado de activación.

- De manera automática se inicializa la máquina en estadoInicio, y se espera en ese estado hasta que el muestreo llega a 15, y no se reciben inputs de T_x . Es entonces que se cambia a estadoDataOut.
- En estadoDataOut, se empieza con el muestreo de 0, y se le suma 1 hasta que llega a 15 nuevamente. En este estado, también se agrega cualquier dato recibido al registro y se le suma 1 a index por cada dato registrado. Cuando index llega a 8 y muestreo a 15, se cambia de estado.
- Por último estadoStop, inicia con un muestreo de 0, y se le suma a este dato, hasta que se vuelve igual a 15. Entonces se reinicia la máquina y vuelve a estadoInicio.

IV. Simulaciones

Se adjuntan las imágenes en el repositorio de Github creado.

V. Funcionamiento de los módulos

El módulo UART_TX es el encargado de realizar la transmisión de datos en el protocolo de UART. Su función principal es tomar datos de 8 bits en formato paralelo (bloques) y convertirlos en una señal serie (uno por uno) para enviarlos a través de una línea de comunicación (Peña & Grace, 2020) .

El proceso de transmisión comienza cuando el sistema indica que hay un dato listo para mandar. En ese momento, el módulo genera un bit de inicio, conocido como start bit. Este se encarga de avisar al receptor que está por comenzar una transmisión de datos. Después de esto, procede a enviar los 8 bits, uno por uno a una velocidad establecida por el baud rate. Una vez finalizada la transmisión, el módulo envía una señal de parada conocida como stop bit que indica el fin de la transmisión (Jawaid, 2026) .

Por otro lado, el módulo UART_RX es el encargado de recibir la señal en serie y reconstruir esta misma a un dato paralelo de 8 bits (Peña & Grace, 2020) . En este caso el módulo permanece en espera hasta detectar el bit de inicio enviado por el transmisor. Una vez detectado, comienza a capturar la señal bit por bit hasta que detecta el stop bit. Al tener los 8 bits los reconstruye a su formato original y activa nuevamente la señal para recibir otro paquete de información.

VI. Conclusiones

Durante el desarrollo de este proyecto se logró diseñar e implementar en Verilog los módulos de transmisión y recepción del protocolo de comunicación UART, así como su testbench y simulaciones. Primeramente se investigó acerca de este protocolo y se buscó tener un amplio entendimiento en él para que luego se pudiera implementar la lógica de funcionamiento en código. El siguiente paso fue realizar este mismo y diseñar un test bench con el objetivo de comprobar que la lógica fuera correcta y que el diseño del programa fuera adecuado. Finalmente se utilizó la herramienta de TinyTapeout en donde se verificó que el diseño cumple con los requisitos de síntesis de la plataforma.

VII. Referencias

- Academia Lab. (2026). Máquina de estados finitos. *Enciclopedia*. Recuperado 7 de junio de 2026, de <https://academia-lab.com/enciclopedia/maquina-de-estados-finitos/>
- Figueroa, F. (2024, 5 enero). RS-232. SENSORICX. Recuperado 7 de junio de 2026, de <https://sensoricx.com/telecomunicaciones/rs-232/>
- Fisicotrónica. (2024, 4 enero). *Máquina de estados: ¿A que nos referimos?* Recuperado 7 de junio de 2026, de <https://fisicotronica.com/maquina-de-estados-nos-referimos/>
- Jawaid, R. (2026). UART Protocol: Understanding Serial Communication for Engineers. Wevolver. <https://www.wevolver.com/article/uart-protocol-understanding-serial-communication-for-engineers>
- Peña, E., & Grace, M. (2020). UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter. <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>
- Sunwoldb, B. (2024, 25 agosto). *UART vs RS232*. Bare Naked Embedded. Recuperado 7 de junio de 2026, de <https://barenakedembedded.com/uart-vs-rs232/>
- Weis, O. (2026, 18 mayo). *Pinout y especificaciones RS232*. Eltima Blog. Recuperado 7 de junio de 2026, de <https://www.eltima.com/es/articles/serial-port-pinout-guide/>

VIII. Apéndice A

- Link a GoogleColab

[Evidencia 2 - Introduccion a la cadena de valor de los semiconductores - Colab](#)

IX. Apéndice B

- Código Módulo T_x

None

```
// Máquina de estados de un transmisor de señales UART (TX)

module UART_TX(
    input clk, rst,
    input [7:0] dataIn, // dato paralelo a transmitir
    input send,         // señal para iniciar la transmisión
    input clkEn,        // activa el clk (baud rate x16)
    output reg tx,      // serial output, lo enviamos a rx
    output reg busy);   // indica que la transmisión está en curso

    // estados de la FSM
    parameter estadoInicio = 2'd0, // espera la señal de envío
               estadoDataIn = 2'd1, // se transmiten los 8 bits de datos
               estadoStop   = 2'd2; // se finaliza la transmisión

    reg [1:0] estadoActual;
    reg [3:0] sample, index; // contador de muestras, índice del bit actual
    reg [7:0] temp;          // registro de los datos a transmitir

    always @(posedge clk or posedge rst) // maquina de estados
    begin
        if (rst) // valor inicial a reestablecer en caso de rst
            begin
                tx          <= 1'd1;
                busy        <= 1'd0;
                estadoActual <= estadoInicio;
                sample      <= 4'd0;
                index       <= 4'd0;
                temp        <= 8'd0;
            end
        else if (clkEn) // la fsm empieza solo si se activa clkEn
            begin
                case (estadoActual)
                    estadoInicio:
                        begin
                            tx = 1'd1; // línea en reposo en alto
                            busy = 1'd0;
                            if (send) // se inicia la transmisión cuando send es activado
                                begin
                                    estadoActual <= estadoDataIn;
                                    temp <= dataIn; // se carga el dato a transmitir
                                    index <= 4'd0;
                                    sample <= 4'd0;
                                    busy <= 1'd1;
                                end
                            else
                                estadoActual <= estadoInicio;
                        end
                    estadoDataIn:
                        begin
                            tx = 1'd0; // línea en reposo en bajo
                            busy = 1'd1;
                            if (index < 8)
                                begin
                                    temp <= dataIn[index];
                                    index <= index + 1;
                                end
                            else
                                estadoActual <= estadoStop;
                        end
                    estadoStop:
                        begin
                            tx = 1'd1; // línea en reposo en alto
                            busy = 1'd0;
                            estadoActual <= estadoInicio;
                        end
                endcase
            end
        end
    end
```

```

        tx <= 1'd0; // se envía el bit de inicio (start bit)
    end
end

estadoDataIn:
begin
    busy = 1'd1;
    tx = temp[index]; // se transmite el bit en el índice indicado
    sample <= sample + 4'd1; // se le suma 1 a sample automáticamente
    if (sample == 4'd15)
    begin
        index <= index + 4'd1; // se le suma 1 a índice
    end
    if (index == 4'd7 && sample == 4'd15) // cambio de estado cuando se
transmiten 8 bits
    begin
        // y el muestreo es 15 de nuevo
        estadoActual <= estadoStop;
        sample <= 4'd0;
    end
end

estadoStop: // estado final donde se cierra la transmisión
begin
    busy = 1'd1;
    tx = 1'd1; // se envía el bit de parada (stop bit)
    sample <= sample + 4'd1;
    if (sample == 4'd15) // se reinicia la máquina cuando el muestreo es 15
    begin
        estadoActual <= estadoInicio;
        sample <= 4'd0; // reiniciamos el muestreo
        busy <= 1'd0; // se indica que la transmisión terminó
    end
end

default: // estado que se activa automáticamente
begin
    estadoActual <= estadoInicio;
    sample <= 4'd0;
    tx <= 1'd1;
end

endcase
end
end

endmodule

```

X. Apéndice C

- Código Módulo R_x

None

```
// Máquina de estados de un receptor de señales UART (RX)

module UART_RX(
    input clk, rst,
    input rx, // serial input, lo recibimos de tx
    input rdyClr, // limpia rdy después de que es leído
    input clkEn, // activa el clk (baud rate x16)
    output reg rdy, // será el indicador de que dataOut tiene un valor válido
    output reg [7:0] dataOut); // outputs recibidos configurados de manera paralela

    // estados de la FSM
    parameter estadoInicio = 2'd0, // se recibe el bit de entrada
        estadoDataOut = 2'd1, // se realiza el muestreo por 8 bits
        estadoStop = 2'd2; // se finaliza el muestreo

    reg [1:0] estadoActual;
    reg [3:0] sample, index; // contador de muestras, índice del bit actual
    reg [7:0] temp; // registro de los datos recibidos de tx

    always @(posedge clk or posedge rst) // maquina de estados
    begin
        if (rst) // valor inicial a reestablecer en caso de rst
            begin
                rdy <= 1'd0;
                dataOut <= 8'd0;
                estadoActual <= estadoInicio;
                sample <= 4'd0;
                index <= 4'd0;
                temp <= 8'd0;
            end

        else if (rdyClr) // se limpia rdy cuando se llama rdyClr
            rdy <= 1'd0;

        else if (clkEn) // la fsm empieza solo si se activa clkEn
            begin
                case (estadoActual)
                    estadoInicio:
                        begin
                            if (!rx || sample != 4'd0) // si no se reciben datos de tx o sample NO es
                                0 a 4 bits
                                sample <= sample + 4'd1; // se le suma 1 al muestreo

                            if (sample == 4'd7 && !rx) // cambio de estado cuando no se reciben datos
                                de tx (rx == 0)
                                begin
                                    // y la muestreo llega 15
                                    estadoActual <= estadoDataOut;
                                    index <= 4'd0;
                                    sample <= 4'd0;
                                    temp <= 8'd0;
                                end
                        end

                    estadoDataOut:
                        begin
                            sample <= sample + 4'd1; // se le suma 1 a sample automaticamente
                            if (sample == 4'd7)
                                begin
```

```

                                temp[index] <= rx; // se agrega el dato recibido al registro, en el
indice indicado
                                index <= index + 4'd1; // se le suma 1 a indice
                                end
                                if (index == 4'd8 && sample == 4'd15) // cambio de estado cuando se
reciben 8 bits
                                begin
                                    estadoActual <= estadoStop; // y el muestreo es 15 de nuevo
                                    sample <= 4'd0;
                                end
                                end

                                estadoStop: // estado final donde se manda el dato
                                begin
                                    if (sample == 4'd7) // se reinicia la maquina cuando el muestro es 15
                                    begin
                                        estadoActual <= estadoInicio;
                                        dataOut <= temp; // se envia el registro de datos recibidos
                                        rdy<= 1'd1; // se activa el indicador
                                        sample <= 4'd0; // reiniciamos el muestreo
                                    end
                                    else
                                        sample <= sample + 4'd1;
                                    end
                                end

                                default: // estado que se activa automaticamente
                                begin
                                    estadoActual <= estadoInicio;
                                    sample <= 4'd0;
                                end
                                endcase
                                end
                                end

                                endmodule

```

XI. Apéndice D

- Código Conjunto

None

// codigo conjunto de UART_TX y UART_RX

```

module UART(
    input      clk,
    input      rst,
    input  [7:0] dataIn, // entrada a TX
    input      send, // señal para iniciar transmisión
    input      clkEn, // activa baud rate (x16)
    input      rdyClr, // limpia la rdy en RX
    output     busy, // TX ocupado
    output     rdy, // RX tiene dato válido

```

```

    output [7:0] dataOut // dato recibido por RX
);

wire serial; // línea serie que conecta TX a RX

UART_TX u_tx ( // como se llaman las variables de TX en este código
    .clk      (clk),
    .rst      (rst),
    .dataIn   (dataIn),
    .send     (send),
    .clkEn    (clkEn),
    .tx       (serial),
    .busy     (busy)
);

UART_RX u_rx ( // como se llaman las variables de RX en este código
    .clk      (clk),
    .rst      (rst),
    .rx       (serial),
    .rdyClr   (rdyClr),
    .clkEn    (clkEn),
    .rdy      (rdy),
    .dataOut  (dataOut)
);
endmodule

module UART_TX( //maquina de estados TX
    input clk, rst, send, clkEn,
    input [7:0] dataIn,
    output reg tx, busy);

    parameter estadoInicio = 2'd0,
               estadoDataIn = 2'd1,
               estadoStop   = 2'd2;

    reg [1:0] estadoActual;
    reg [3:0] sample, index;
    reg [7:0] temp;

    always @(posedge clk or posedge rst) // maquina de estados
    begin
        if (rst) // estado inicial al cual volver en caso de rst
        begin
            tx          <= 1'b1;
            busy        <= 1'b0;
            estadoActual <= estadoInicio;
            sample      <= 4'd0;
            index       <= 4'd0;
            temp        <= 8'd0;
        end

        else if (clkEn) // solo avanza con el baud rate
        begin
            case (estadoActual)
                estadoInicio:
                begin
                    tx <= 1'b1;
                    busy <= 1'b0;

```



```

        if (send)
        begin
            estadoActual <= estadoDataIn;
            temp <= dataIn;
            index <= 4'd0;
            sample <= 4'd0;
            busy <= 1'b1;
            tx <= 1'b0;
        end
    end

    estadoDataIn:
    begin
        busy <= 1'b1;
        tx <= temp[index];
        sample <= sample + 4'd1;
        if (sample == 4'd15)
            index <= index + 4'd1;
        if (index == 4'd7 && sample == 4'd15)
        begin
            estadoActual <= estadoStop;
            sample <= 4'd0;
        end
    end

    estadoStop:
    begin
        busy <= 1'b1;
        tx <= 1'b1;
        sample <= sample + 4'd1;
        if (sample == 4'd15)
        begin
            estadoActual <= estadoInicio;
            sample <= 4'd0;
            busy <= 1'b0;
        end
    end

    default:
    begin
        estadoActual <= estadoInicio;
        sample <= 4'd0;
        tx <= 1'b1;
    end
endcase
end
end
endmodule

module UART_RX(
    input clk, rst, rx, rdyClr, clkEn,
    output reg rdy,
    output reg [7:0] dataOut
);
    parameter estadoInicio = 2'd0,
               estadoDataOut = 2'd1,
               estadoStop = 2'd2;

```

```

reg [1:0] estadoActual;
reg [3:0] sample, index;
reg [7:0] temp;

always @(posedge clk or posedge rst)
begin
    if (rst) // estado inicial al cual volver encaso de rst
    begin
        rdy          <= 1'b0;
        dataOut       <= 8'd0;
        estadoActual <= estadoInicio;
        sample        <= 4'd0;
        index         <= 4'd0;
        temp          <= 8'd0;
    end
    else if (rdyClr) // limpiar rdy
        rdy <= 1'b0;
    else if (clkEn) // avanza con el baud rate
    begin
        case (estadoActual)
            estadoInicio:
            begin
                if (!rx || sample != 4'd0)
                    sample <= sample + 4'd1;
                if (sample == 4'd7 && !rx)
                begin
                    estadoActual <= estadoDataOut;
                    index        <= 4'd0;
                    sample       <= 4'd0;
                    temp         <= 8'd0;
                end
            end
            estadoDataOut:
            begin
                sample <= sample + 4'd1;
                if (sample == 4'd7)
                begin
                    temp[index] <= rx;
                    index      <= index + 4'd1;
                end
                if (index == 4'd8 && sample == 4'd15)
                begin
                    estadoActual <= estadoStop;
                    sample      <= 4'd0;
                end
            end
            estadoStop:
            begin
                if (sample == 4'd7)
                begin
                    estadoActual <= estadoInicio;
                    dataOut      <= temp;
                    rdy          <= 1'b1;
                    sample      <= 4'd0;
                end
            end
        endcase
    end
end

```

```

        sample <= sample + 4'd1;
    end

    default:
    begin
        estadoActual <= estadoInicio;
        sample <= 4'd0;
    end
endcase
end
end
endmodule

```

XII. Apéndice E

- Código Generador de Baud Rate

```

None
// generador del baud rate

module BAUD_RATE(
    input  clk,
    input  rst,
    output txEnb, // activa el transmisor
    output rxEnb // activa el receptor
);
    reg [12:0] txCounter;
    reg [9:0]  rxCounter;

    always @(posedge clk) begin // baud rate base
        if (rst || txCounter == 5208)
            txCounter <= 13'h0;
        else
            txCounter <= txCounter + 1'b1;
        end

    always @(posedge clk) begin // baud rate mas rapido (x16)
        if (rst || rxCounter == 325)
            rxCounter <= 10'h0;
        else
            rxCounter <= rxCounter + 1'b1;
        end

    assign txEnb = (txCounter == 0) ? 1'b1 : 1'b0; // si rxCounter=0, se manda un 1, sino 0
    assign rxEnb = (rxCounter == 0) ? 1'b1 : 1'b0; // si rxCounter=0, se manda un 1, sino 0

endmodule

```

XIII. Apéndice F

- Código de Prueba

```
None
// Testbench para el DUT de UART (TX + RX)

module UART_tb();
    reg clk, rst; // se establecen los inputs del código a probar como registros
    reg [7:0] dataIn;
    reg send;
    reg clkEn;
    reg rdyClr;
    wire busy; // se establecen los outputs como wires
    wire rdy;
    wire [7:0] dataOut;
    wire txEnb;
    wire rxEnb;

    BAUD_RATE u_baud( // variables del baud rate en el tb .variable_tb(variable del diseño)
        .clk (clk),
        .rst (rst),
        .txEnb(txEnb),
        .rxEnb(rxEnb)
    );

    UART DUT( // establecemos la variable del diseño como en el tb como:
        .variable_tb(variable del diseño)
        .clk (clk),
        .rst (rst),
        .dataIn (dataIn),
        .send (send),
        .clkEn (clkEn),
        .rdyClr (rdyClr),
        .busy (busy),
        .rdy (rdy),
        .dataOut(dataOut)
    );

    always #5 clk = ~clk; // cuantos tiempos se utilizaran en clk = 1, y cuantos en clk = 0

    initial // se prueban posibles configuraciones del código
    begin
        clk = 0; // inicializamos todas las variables en 0
        rst = 0;
        send = 0;
        rdyClr = 0;
        dataIn = 8'd0;

        #20; // se le dan 20 ciclos para correr el código
        rst = 1; // se reinicia el UART
        #20;
        rst = 0;

        #200; // se envia 10100101
```

```

    dataIn = 8'hA5; // transmite el dato
    send   = 1; // se activa la señal de envío
    @(posedge busy); // espera a que TX acepte el dato
    send   = 0;
    @(negedge busy); // espera a que TX termine la transmisión completa
    #500; // vscode lo pone como RGB pero no afecta

    dataIn = 8'hFF; // se envía 11111111
    send   = 1;
    @(posedge busy);
    send   = 0;
    @(negedge busy);
    #500;

    dataIn = 8'h00; // se envía 00000000
    send   = 1;
    @(posedge busy);
    send   = 0;
    @(negedge busy);
    #500;

    @(posedge txEnb);
    dataIn = 8'h3C; // dato de prueba 00111100
    send   = 1;
    @(posedge busy);
    send   = 0;
    @(negedge busy);
    #500;
    rdyClr = 1; // se limpia la señal rdy
    #10;
    rdyClr = 0;

    $finish;
end

initial
begin
    $monitor("Cambio en señales UART: dataIn=%h send=%b busy=%b rdy=%b dataOut=%h",
            dataIn, send, busy, rdy, dataOut);
end

initial
begin
    $dumpfile("UART.vcd"); // genera un archivo para las señales analizadas
    $dumpvars(0, UART_tb); // establece en que tiempo empieza la simulación, y las
variables a utilizar
end
endmodule

```